现代智能体 CLI

第05讲 · 2026年9月15日

layout: default

当前位置

Lab01 留下了一个未解决的问题 {.assert}

你按谁选择了每个部分对提示词进行了排序。

提示词部分	选择者
系统提示词	Aider，按模式
添加和只读文件	你，按命令
代码库地图	Aider，按排名
历史记录	你开始，Aider 修剪它
你的消息	你

layout: default

Aider 是其中之一

Aider、模型与代码库



layout: default

循环，写出来

六个步骤，按顺序

1 组装
你和 Aider，按表格
2 回答
模型。文本输出
3 解析
Aider，按固定规则
4 应用
Aider，否则它拒绝
5 修复

layout: default

回环的那一个

修复是插入，不是阶段

- 不匹配的块是**正常的**，不是例外
- Aider 将失败写回作为提示词，步骤2再次运行
- 最多三次，然后它放弃并告诉你

它在步骤2重新进入。它从不选择不同的步骤，也从不选择不同的文件。

layout: statement

谁选择 *下一个动作*？

layout: section

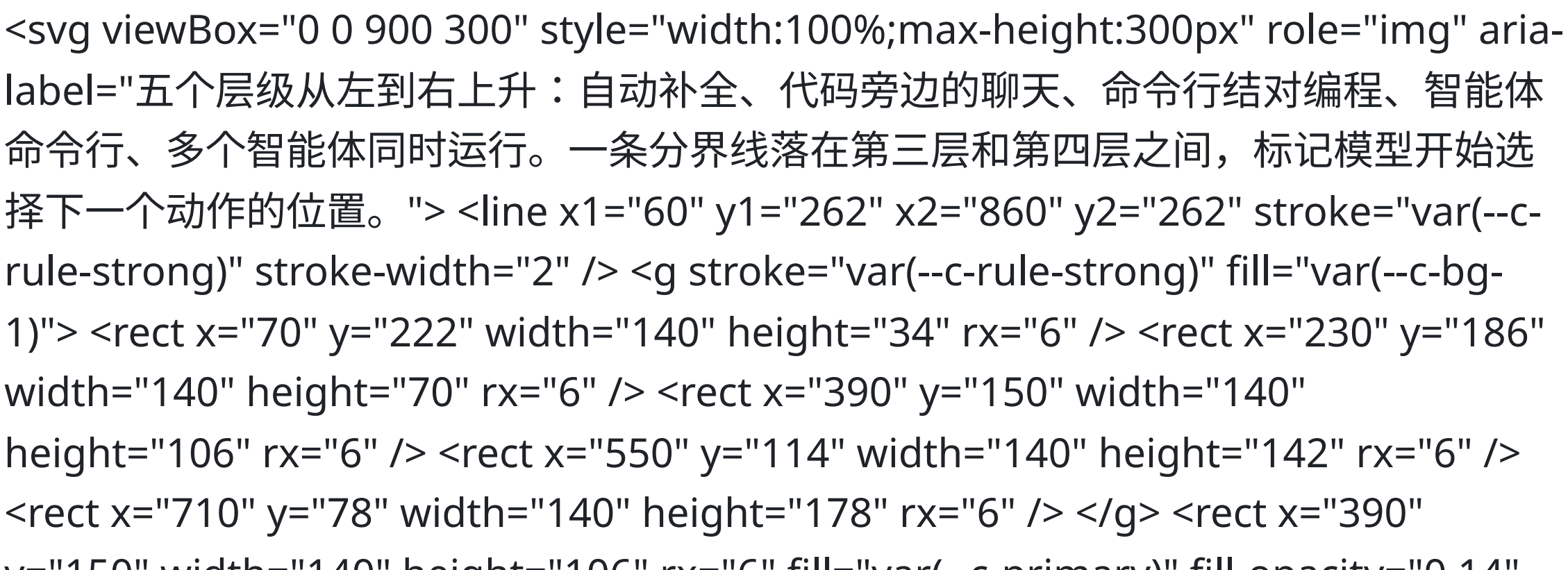
自主性梯度

五个层级，以及你一直站在哪里

layout: default

梯度

五个层级，不是排名 {.assert}



layout: default

第3层

你在第3层 {.assert}

	在第3层
你移交	一个有界编辑任务
你选择	文件，以及任务在哪里结束
它自动化	匹配、修复、测试、提交
你检查	在任务之间

在你划定的边界内的真实自动化。

layout: default

权衡

更高的层级花费更多来问更少 {.assert}

<ph-trend-up-bold class="text-3xl text-blue-600 shrink-0" />
单位工作的成本
一次搜索、读取五个文件并三次运行测试的运行比一个有界编辑花费更多的注意力。
<ph-trend-down-bold class="text-3xl text-amber-600 shrink-0" />
你自己做的决定
每个完成结果的决策更少，而且你在每个结果上花费的注意力更少。

layout: default

选择一个层级

第3层适合你可以命名的改动 {.assert}

一个你可以指着的文件中的两行修复在第3层更便宜、更安全。
为它支付第4层的价格买不到任何东西。

技能是选择层级，而你无法选择你看不到的东西。

layout: section

什么让工具具有智能体特性

四个原语，以及一个你可以应用的测试

layout: default

定义

四个原语，不是一种感觉 {.assert}

<ph-arrows-clockwise-bold class="text-2xl text-blue-600 mb-2" />
循环
无需询问就采取另一步骤。
<ph-wrench-bold class="text-2xl text-blue-600 mb-2" />
工具
超出发出文本的动作。
<ph-list-checks-bold class="text-2xl text-amber-600 mb-2" />
规划
将目标分解为步骤。

layout: default

测试

Aider 拥有全部四个 {.assert}

原语	它在 Aider 中已经在哪里
循环	失败的编辑变成下一个提示词；测试可以反馈
工具	读取、写入、提交、运行你配置的命令
规划	用散文起草的计划，停下来供人阅读
记忆	会话历史，加上你从未添加过的代码的代码库地图

layout: statement

区别在于*谁选择*。

layout: default

移动

一个选择改变了层级 {.assert}

在第3层，你选择文件、边界和是否运行计划。

将下一个动作的选择从工具链移到模型，
人们觉得关于智能体工具令人惊讶的一切都随之而来。

一个改变。不是一个新的智能类别。

layout: section

聊天休息

五分钟，关于你已经知道的工具

layout: default

聊天休息 · 5 分钟

放弃一个选择 {.assert}

两周的 taskr Aider 使用。每次仍然有三个选择是你的：
它看到哪些文件，任务在哪里结束，计划是否运行。

将**正好一个**交给模型。

然后说在你放心让它在你关心的代码库上运行之前你必须构建什么。 移交和护栏哪个工作量更大？

layout: section

PollEv

https://PollEv.com/marcusdarden365

layout: section

同样的请求，上一层级

现场，在这台机器上

layout: default

请求

你知道这在第3层花了什么 {.assert}

```
Add a --priority flag to taskr's add command.
```

你添加了三个文件，因为你没有添加的文件不在上下文中。
你框定了任务。你阅读了 diff。你自己发现了缺失的写入。

layout: default

现场演示

没有人需要输入 /add {.assert}

它搜索
找到参数解析的位置，无需被告知。
它读取
跟踪代码到实际触及的文件。
它运行
执行测试并对它们所说的内容做出反应。
上下文组装变成了程序通过行动做的事情。

layout: default

追踪记录

每个结果选择下一步 {.assert}

```
search "add_argument"   → three hits, one in cli.py
read cli.py             → finds the parser
read store.py           → finds the save path
edit cli.py, task.py, store.py
run tests               → 1 failed
edit store.py
run tests               → passed
```

那个顺序没有任何人提前写下来。

layout: default

诚实

今天的运行是关于今天的事实 {.assert}

你刚刚看到的是这个模型和这个构建在这个代码库上，
今天下午所做的事情。

运行糟糕比运行顺利更有用。不要把它救得太快。

layout: section

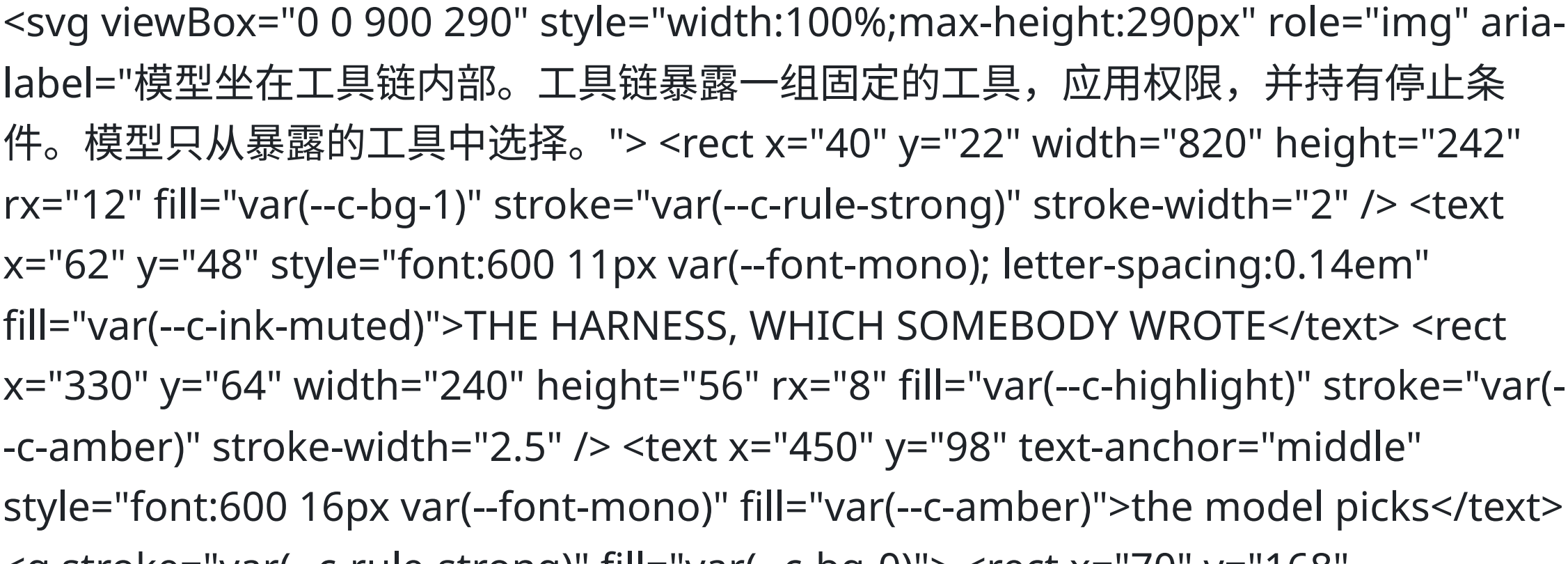
工具链仍然决定什么

演示隐藏的部分

layout: default

菜单

模型从一个它没有编写的菜单中选择 {.assert}



layout: default

安全性

权限才是真正的安全层 {.assert}

你在实验课中被 handed 了这个形状。在人类批准之前，
磁盘上没有任何改变，无论模型说什么都成立，因为
程序强制执行它，而不是提示词。

同一个问题扩大到更大规模：当模型可以运行命令时，哪些命令可以无需询问就运行？

layout: statement

文字询问。代码和权限 *强制执行*。

layout: default

停止

循环需要一个停止的理由 {.assert}

一个它可以识别为已满足的目标。一个它可以花费的预算。一个中断的人类。

在第3层，任务边界是停止条件，你每次都手工
划定它。

layout: default

你的项目构建

第3层是重点，不是限制 {.assert}

精确匹配。任何写入之前的一次批准。每个接受的编辑集一个提交。受保护的撤销。

这些使得失败后的状态成为你可以用一句话说出来并在测试中断言的东西。

构建那个，你就构建了循环在它下面需要的东西。

layout: default

习惯

追踪记录就是证据 {.assert}

当运行产生错误结果时，输出不会告诉你为什么。

哪个步骤选择了错误的文件？它误读了哪个结果？它 improvisation 了哪个缺失的工具？

"模型不擅长这个"是人们得到的答案，而且很少是证据支持的答案。

layout: default

本周

项目构建三周后截止 {.assert}

时间	事项
9月17日周四	L06 结束 Apply
9月21日周一 / 9月22日周二	Lab02，支持的构建时间
9月24日周四，晚上	Hackathon 1，你自己的项目
10月6日周二，晚上11:59	项目构建截止

layout: default

带走这些

三个问题

1. 你默认伸手去拿的工具在梯度上的什么位置，是你选择的还是继承的？
2. 在你让模型无需询问就运行命令之前必须发生什么？
3. 你需要在一段追踪记录中看到什么才会相信一个运行失败的原因不是模型？

layout: statement

周四，循环不再*是别人的*。

layout: end